

EXAM POV NOTES (Q&A FORMAT)**10 MARKS QUESTIONS**

Q1. Explain the A* algorithm in detail with its complete steps. Illustrate with the 8-puzzle problem using misplaced tiles as a heuristic function.

A* is an informed search algorithm that uses both the actual cost from start and estimated cost to goal for finding optimal paths efficiently.

- **Algorithm Overview:** A* combines the advantages of uniform cost search and greedy best-first search by using evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is actual cost from start and $h(n)$ is heuristic estimate to goal.
- **Initialization Step:** Set OPEN = {s} containing start state, CLOSED = {} as empty set. Initialize $g(s) = 0$ (no cost to reach start) and $f(s) = h(s)$ (only heuristic estimate initially).
- **Failure Condition:** If OPEN becomes empty without finding goal, terminate and report failure, meaning no solution path exists from start to any goal state.
- **Selection Step:** Select the state n with minimum $f(n)$ value from OPEN list. This state is most promising based on combined actual and estimated costs. Move n to CLOSED list.
- **Termination Condition:** If selected state n belongs to goal set G, terminate successfully and return $f(n)$ as the optimal path cost from start to goal.
- **Expansion Step:** Generate all successors m of state n using available operators. For each successor, calculate costs and decide whether to add to OPEN or update existing values.
- **Handling New States:** If successor m is not in OPEN or CLOSED, set $g(m) = g(n) + C(n,m)$ where $C(n,m)$ is edge cost. Calculate $f(m) = g(m) + h(m)$ and insert m into OPEN.
- **Handling Existing States:** If m already exists in OPEN or CLOSED, update $g(m) = \min\{g(m), g(n) + C(n,m)\}$. Recalculate $f(m)$. If $f(m)$ decreased and m is in CLOSED, move m back to OPEN for re-expansion.
- **8-Puzzle Example Setup:** Consider initial state with tiles arranged randomly and goal state with tiles 1-8 in order with blank at position 9. Use misplaced tiles heuristic: $h(n) = \text{number of tiles not in goal position (excluding blank)}$.
- **8-Puzzle Execution:** Example: Initial state has 5 misplaced tiles, so $h(s)=5$, $g(s)=0$, $f(s)=5$. Expand by moving blank in 4 directions. For each successor, count misplaced tiles, add move cost (1), calculate f value, and select minimum f state from OPEN. Repeat until goal configuration reached.

Q2. Compare and contrast Breadth-First Search, Depth-First Search, and Uniform Cost Search in terms of their time complexity, space complexity, completeness, and optimality.

BFS, DFS, and UCS are fundamental uninformed search strategies that differ significantly in their resource usage and solution guarantees.

- **Breadth-First Search Overview:** BFS explores all nodes at depth d before moving to depth d+1, using FIFO queue. Systematically expands level by level ensuring shortest path in terms of number of steps.
- **BFS Complexity:** Time Complexity: $O(b^d)$ where b is branching factor and d is goal depth, as it explores all nodes up to goal level. Space Complexity: $O(b^d)$ as it stores all nodes at current level in OPEN.
- **BFS Completeness and Optimality:** Complete: Always finds solution if one exists at finite depth. Optimal: Finds shortest path only when all operators have equal cost (unit cost). Not optimal for varying edge costs.
- **Depth-First Search Overview:** DFS explores deepest node first using LIFO stack, going as deep as possible before backtracking. Memory efficient but may get trapped in infinite branches.

- **DFS Complexity:** Time Complexity: $O(b^m)$ where m is maximum depth of state space, potentially much larger than d . Space Complexity: $O(bm)$ storing only nodes on current path, much better than BFS.
- **DFS Completeness and Optimality:** Complete: Only in finite state spaces without cycles; may loop infinitely otherwise. Optimal: Not guaranteed as it returns first solution found, which may not be shortest or least cost.
- **Uniform Cost Search Overview:** UCS expands node with lowest path cost $g(n)$ from start, using priority queue ordered by cumulative cost. Generalizes BFS for non-uniform operator costs.
- **UCS Complexity:** Time Complexity: $O(b^{(C^*/\epsilon)})$ where C^* is optimal cost and ϵ is minimum operator cost. Space Complexity: Similar to time complexity as all generated nodes stored.
- **UCS Completeness and Optimality:** Complete: Yes, if all operator costs are positive and solution exists. Optimal: Always finds minimum cost path when all costs are non-negative, guaranteed by expanding in order of increasing cost.
- **Comparative Summary:** BFS best for unweighted graphs needing shortest path; DFS best when memory limited or solutions are deep; UCS best for weighted graphs needing optimal cost path. Choose based on problem constraints and resource availability.

Q3. Describe the Missionaries and Cannibals problem in detail. Formulate it as a state space search problem with state representation, operators, start state, and goal state. Provide the complete solution path.

The Missionaries and Cannibals is a classic AI problem involving constraint satisfaction and state space search with safety constraints.

- **Problem Statement:** Three missionaries and three cannibals must cross a river using a boat that holds maximum two people. Constraint: missionaries cannot be outnumbered by cannibals on either bank, or they will be eaten.
- **State Representation:** State = $(\#m, \#c, b)$ where $\#m$ = number of missionaries on left bank, $\#c$ = number of cannibals on left bank, b = boat position (0=left bank, 1=right bank). Right bank has $(3-\#m, 3-\#c)$.
- **Start and Goal States:** Start State: $(3, 3, 0)$ - all 3 missionaries, 3 cannibals, and boat on left bank. Goal State: $(0, 0, 1)$ - all missionaries, cannibals crossed to right bank, boat on right side.
- **Operators Definition:** Five possible boat trips: $(1,0)$ one missionary, $(0,1)$ one cannibal, $(1,1)$ one of each, $(2,0)$ two missionaries, $(0,2)$ two cannibals. Each moves people between banks with boat.
- **Constraint Validation:** After each operator application, check constraints: on left bank $\#m \geq \#c$ or $\#m=0$; on right bank $(3-\#m) \geq (3-\#c)$ or $(3-\#m)=0$. Invalid states pruned from search.
- **Solution Path Step 1-2:** $(3,3,0) \rightarrow$ Move $(0,2)$: $(3,1,1)$ - two cannibals cross right. Then $(3,1,1) \rightarrow$ Move $(0,1)$: $(3,2,0)$ - one cannibal returns left. State $(3,2,0)$ is valid.
- **Solution Path Step 3-4:** $(3,2,0) \rightarrow$ Move $(0,2)$: $(3,0,1)$ - two cannibals cross right, all cannibals now right. Then $(3,0,1) \rightarrow$ Move $(0,1)$: $(3,1,0)$ - one cannibal returns left.
- **Solution Path Step 5-6:** $(3,1,0) \rightarrow$ Move $(2,0)$: $(1,1,1)$ - two missionaries cross right. Then $(1,1,1) \rightarrow$ Move $(1,1)$: $(2,0,0)$ - one missionary and one cannibal return left.
- **Solution Path Step 7-8:** $(2,0,0) \rightarrow$ Move $(2,0)$: $(0,0,1)$ - two missionaries cross to right, reaching goal state. Total moves: 7 river crossings (excluding goal state).
- **Search Space Characteristics:** State space has multiple valid paths but many dead-ends due to constraints. Search algorithms like BFS guarantee finding shortest solution (11 states in optimal path including start). Demonstrates importance of constraint checking in state generation.

Q4. Explain Branch-and-Bound algorithm with its complete procedure. How does it differ from Uniform Cost Search? Discuss its advantages and disadvantages.

Branch-and-Bound is an optimization algorithm that systematically explores state space while maintaining upper bound on solution cost to prune unpromising branches.

- **Algorithm Initialization:** Set OPEN = {s} with start state, CLOSED = {} empty. Initialize $C(s) = 0$ for start state cost and $C^* = \infty$ as initial upper bound representing best solution cost found so far.
- **Primary Termination:** If OPEN becomes empty, terminate and return C^* as the optimal solution cost. This means all possible paths have been explored and best solution identified.
- **State Selection:** Select a state n from OPEN and move it to CLOSED. Unlike UCS which always selects minimum cost, Branch-and-Bound can use various selection strategies (best-first, depth-first, etc.).
- **Goal State Check:** If selected state n is a goal state ($n \in G$) and $C(n) < C^*$, update upper bound $C^* = C(n)$. This new bound helps prune future branches. Continue search rather than terminating.
- **Pruning Condition:** Expand state n only if $C(n) < C^*$, otherwise skip expansion. This pruning eliminates exploring branches that cannot lead to better solutions than current best, saving computational effort.
- **Successor Generation:** For states passing pruning test, generate all successors m . Calculate $C(m) = C(n) + C(n,m)$ for new states and add to OPEN with their costs.
- **Handling Duplicate States:** If successor m already exists in OPEN or CLOSED, update $C(m) = \min\{C(m), C(n) + C(n,m)\}$. If cost decreased and m in CLOSED, move back to OPEN for potential re-expansion.
- **Difference from UCS:** UCS terminates upon first goal discovery with guaranteed optimal cost. Branch-and-Bound continues exploring to find all solutions, maintaining best cost found. UCS suitable for single optimal path; B&B for optimization problems needing best solution verification.
- **Advantages:** Memory efficient as it can use depth-first strategies; finds globally optimal solution by exhaustive bounded search; pruning reduces exploration significantly; suitable for optimization problems with multiple solutions.
- **Disadvantages:** Slower than UCS as it may explore multiple goal states; requires good initial bound for effective pruning; may degenerate to exhaustive search with poor bounds; not suitable for infinite state spaces without additional heuristics.

Q5. What is an admissible heuristic? Prove that if the heuristic function is admissible, A* algorithm always finds an optimal solution. Discuss the conditions under which A* terminates.

Admissible heuristics are essential for A* optimality, ensuring the algorithm never overestimates the cost to reach goal states.

- **Admissible Heuristic Definition:** A heuristic $h(n)$ is admissible if it never overestimates the actual minimum cost to reach goal from state n . Formally: $h(n) \leq f^*(n)$ for all states n , where $f^*(n)$ is true minimum cost to goal.
- **Examples of Admissible Heuristics:** For 8-puzzle: misplaced tiles heuristic (counts tiles not in goal position) is admissible as each tile needs at least one move. Manhattan distance (sum of horizontal and vertical distances) also admissible and more informed.
- **Proof Setup - Optimal Path Property:** Assume A* using admissible $h(n)$ terminates at goal state G . Let's prove G must be on optimal path. Suppose not, and let G' be actual optimal goal with $f^*(s) < f(G)$.

- **Proof - State on Optimal Path:** At termination, there must exist unexpanded state n on optimal path to G' in OPEN with $f(n) = g(n) + h(n)$. Since h is admissible, $h(n) \leq f^*(n)$, so $f(n) \leq g(n) + f^*(n) = f^*(s)$.
- **Proof - Contradiction:** Since G was selected over n , we have $f(G) \leq f(n)$. But $f(G) = g(G)$ as $h(goal)=0$. This gives $g(G) \leq f(n) \leq f^*(s) < f(G)$, a contradiction. Therefore, G must be optimal.
- **Optimality Guarantee:** This proves A* with admissible heuristic always selects optimal goal first. Cannot terminate at suboptimal goal because optimal path state always has lower f value in OPEN.
- **Termination Condition 1 - Finite State Spaces:** For finite state spaces with admissible heuristic, A* always terminates. CLOSED grows monotonically, OPEN contains finite states, and optimal path exists in space.
- **Termination Condition 2 - Infinite State Spaces:** For infinite state spaces, A* terminates if there exists a path from start to goal. The algorithm will eventually select goal state as f values along optimal path remain bounded below $f^*(s)$.
- **Termination Condition 3 - No Solution Case:** If no path exists from start to goal, A* may not terminate in infinite spaces. It will keep expanding states indefinitely unless state space exhausted (finite case).
- **Consistency and Additional Properties:** A stronger property is consistency (monotonicity): $h(n) \leq C(n,m) + h(m)$ for all successors m . Consistent heuristics are admissible. With consistency, A* expands states in order of increasing f values, and no state needs re-expansion from CLOSED.

5 MARKS QUESTIONS

Q6. Explain the 8-Queens problem with at least two different formulations. Compare the state space size and efficiency of these formulations.

The 8-Queens problem requires placing 8 queens on 8×8 chessboard such that no queen attacks another, demonstrating how problem formulation affects search efficiency.

- **Problem Description:** A queen attacks any piece on same row, column, or diagonal. Goal: place 8 queens where none attack each other. Classic constraint satisfaction problem with 92 distinct solutions.
- **Formulation I - Any Arrangement:** State: any arrangement of 0 to 8 queens anywhere on 64 squares. Operators: add queen to any empty square. State space: enormous, $C(64,0) + C(64,1) + \dots + C(64,8) \approx 10^{14}$ states.
- **Formulation I - Inefficiency:** Most states invalid with attacking queens. Search explores many illegal configurations. Extremely inefficient due to massive branching factor (up to 64 choices per state) and late constraint checking.
- **Formulation II - Non-attacking Constraint:** State: arrangement of 0-8 queens with none attacked. Operators: place queen in leftmost empty column ensuring no attacks. State space: significantly reduced as only valid placements allowed.
- **Formulation II - Efficiency Advantage:** Each operator checks constraints before adding queen, pruning invalid branches early. Branching factor reduced to ~8 (rows per column). Reaches solution much faster with reduced state space exploration.

Q7. Describe Iterative Deepening search. How does it achieve a balance between time and space complexity? Compare it with Bi-directional search.

Iterative Deepening combines BFS optimality with DFS space efficiency, making it practical for large state spaces with unknown goal depth.

- **Iterative Deepening Mechanism:** Performs depth-limited DFS repeatedly with increasing depth limits (0, 1, 2, ..., d). Each iteration explores completely up to current depth limit before incrementing limit and restarting from start state.
- **Time Complexity Analysis:** Time: $O(b^d)$ - appears wasteful as states re-expanded, but dominated by last level. Nodes at depth d expanded once, d-1 twice, etc.
Total: $(d)b^d + (d-1)b^{d-1} + \dots = O(b^d)$, same as BFS asymptotically.
- **Space Complexity Advantage:** Space: $O(bd)$ - only stores nodes on current path from root to depth-limited frontier, like DFS. Massive improvement over BFS's $O(b^d)$ space, enabling solving much larger problems.
- **Optimality and Completeness:** Complete: finds solution if exists at finite depth. Optimal: like BFS, finds shallowest (shortest path) solution when operators have uniform cost. Combines best properties of BFS and DFS.
- **Bi-directional Search Comparison:** Bi-directional runs two simultaneous searches: forward from start, backward from goal, meeting in middle. Requires reversible operators.
Time: $O(b^{d/2})$ and Space: $O(b^{d/2})$, better than iterative deepening but needs more memory than ID and cannot always reverse operators.

Q8. What are OPEN and CLOSED lists in search algorithms? Explain their role in avoiding repeated states and ensuring algorithm termination with an example.

OPEN and CLOSED lists are fundamental data structures in graph search that manage state exploration and prevent redundant work.

- **OPEN List Purpose:** OPEN contains generated but unexpanded states, representing frontier of search. Organized based on search strategy: FIFO queue for BFS, LIFO stack for DFS, priority queue for A* and UCS.
- **CLOSED List Purpose:** CLOSED stores already expanded states to prevent re-expansion. Acts as visited set, ensuring each state processed only once, preventing infinite loops in cyclic state spaces.
- **Avoiding Repeated States:** Before adding successor to OPEN, check if already in OPEN or CLOSED. If duplicate found, either skip or update cost if new path cheaper. Prevents exponential explosion from exploring same state via different paths.
- **Ensuring Termination:** In finite state spaces, CLOSED grows while OPEN eventually empties as states move from OPEN to CLOSED. When OPEN empty, all reachable states explored, guaranteeing termination with failure if no goal found.
- **Example - Simple Graph:** Consider states {A,B,C,D} where $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$. Start A, goal D. Initially $OPEN=\{A\}$, $CLOSED=\{\}$. Expand A: $OPEN=\{B,C\}$, $CLOSED=\{A\}$. Expand B: generate D, also C but C in OPEN so skip. $OPEN=\{C,D\}$, $CLOSED=\{A,B\}$. Expand C: generate D but already in OPEN. $OPEN=\{D\}$, $CLOSED=\{A,B,C\}$. Expand D: goal found. Without CLOSED, would repeatedly cycle through states.

Q9. Explain Uniform Cost Search algorithm. Under what conditions does it guarantee finding the minimum cost path? What happens if negative operator costs are allowed?

Uniform Cost Search generalizes BFS for weighted graphs, systematically exploring paths in order of cumulative cost from start state.

- **Algorithm Description:** Maintains OPEN as priority queue ordered by path cost $C(n)$ from start. Always expands unexpanded state with lowest cumulative cost $g(n)$, ensuring optimal exploration order.
- **Cost Update Mechanism:** When successor m generated: if new, set $C(m)=C(n)+C(n,m)$ and add to OPEN. If m exists, update $C(m)=\min\{C(m), C(n)+C(n,m)\}$. If cost decreased and m in CLOSED, move back to OPEN.
- **Optimality Guarantee Conditions:** Guarantees minimum cost path when all operator costs are non-negative (≥ 0). Works because states expanded in order of increasing cost, so first time goal reached is via optimal path.
- **Completeness:** Complete if all operator costs positive and solution exists at finite cost. Each expansion increases or maintains minimum path cost in OPEN, ensuring progress toward goal.
- **Negative Costs Problem:** With negative costs, UCS fails. A state already in CLOSED might later be reached via path with negative edges yielding lower total cost. Algorithm doesn't re-expand CLOSED states by default, missing optimal path. Bellman-Ford algorithm needed for negative edges instead.

Q10. Define the basic search problem with its four components [S, s, O, G]. Illustrate each component using the 8-puzzle problem as an example.

The basic search problem provides a formal framework for representing problem-solving tasks as state space exploration with defined start, goal, and transitions.

- **Component S - State Space:** S is the complete set of all possible states in the problem domain. States can be implicitly defined (not enumerated) but must be recognizable. Represents entire problem universe.
- **8-Puzzle State Space S:** For 8-puzzle: S consists of all possible configurations of 8 numbered tiles and one blank on 3×3 grid. Total: $9! = 362,880$ possible arrangements, though only half (181,440) are reachable from any given start state due to parity constraints.
- **Component s - Start State:** s is the specific initial state where problem-solving begins. Must be well-defined and belong to state space S. Represents the given problem instance.
- **8-Puzzle Start State s:** Given specific initial configuration of tiles, for example: [2,8,3,1,6,4,7,blank,5] representing particular scrambled arrangement. This is the starting point of search.
- **Component O - Operators:** O is the set of state transition operators (actions) that transform one state into another. Defines legal moves in problem space. Operators implicitly define graph structure over states.

Q11. Discuss the results and properties of A* algorithm. Why is it considered optimal and complete when the heuristic is admissible?

A* is celebrated for its theoretical guarantees of optimality and completeness under certain conditions, making it the gold standard for informed search.

- **Admissibility Requirement:** A heuristic $h(n)$ is admissible if $h(n) \leq f^*(n)$ for all n , where $f^*(n)$ is true minimum cost to goal. This means never overestimating distance, being optimistic about remaining cost.

- **Finite State Space Termination:** For finite state spaces with admissible heuristic, A* always terminates. Cannot get stuck in infinite loop as CLOSED list grows and state space is bounded.
- **Optimal Path Property:** At any time before termination, OPEN contains at least one state n on optimal path from start s to goal with $f(n) \leq f^*(s)$. This ensures optimal path never completely pruned.
- **Optimality Proof Key Insight:** A* expands states in order of increasing f values. When goal selected, any unexpanded state on better path would have lower f value, contradicting selection. Therefore, first goal reached is optimal.
- **Completeness in Infinite Spaces:** If path exists from start to goal, A* terminates even in infinite state spaces. Algorithm eventually selects goal state as f values along optimal path remain bounded. Without solution path, may not terminate in infinite spaces.

Q12. Explain the concept of state space search. Differentiate between informed (heuristic) search and uninformed (blind) search with examples.

State space search is the foundational paradigm in AI for problem-solving, representing problems as graphs of states connected by operators.

- **State Space Concept:** State space is implicit graph where nodes are problem states and edges are operator applications. Search navigates this graph from start state to goal state(s) finding solution path.
- **Uninformed (Blind) Search:** Uninformed search has no domain knowledge beyond problem definition. Explores systematically using only state space structure. Strategies: BFS explores breadth-first, DFS depth-first, UCS by path cost.
- **Uninformed Search Examples:** BFS for shortest path in unweighted graph, DFS for maze solving, UCS for shortest path in weighted graph. No knowledge about goal location, purely systematic exploration.
- **Informed (Heuristic) Search:** Informed search uses domain-specific knowledge via heuristic function $h(n)$ estimating cost from state n to goal. Guides search toward promising directions, reducing exploration.
- **Informed Search Examples:** A* for 8-puzzle using Manhattan distance heuristic, greedy best-first using straight-line distance in route planning. Heuristics dramatically reduce states explored compared to uninformed methods, making complex problems tractable.

Q13. Describe the water jug problem formulation. Define the state representation, operators, start state, and goal state for this problem.

The water jug problem is a classic state space search problem involving capacity constraints and liquid transfer operations.

- **Problem Statement:** Given two jugs with capacities $C1$ and $C2$ gallons (e.g., 4 and 3 gallons), and unlimited water supply, measure exactly G gallons (e.g., 2 gallons) in one jug.
- **State Representation:** State = (x, y) where x = current water in jug 1 ($0 \leq x \leq C1$) and y = current water in jug 2 ($0 \leq y \leq C2$). State space size: $(C1+1) \times (C2+1)$ possible states.
- **Operators Definition:** Six operators: (1) Fill jug 1 completely, (2) Fill jug 2 completely, (3) Empty jug 1, (4) Empty jug 2, (5) Pour jug 1 into jug 2 until jug 2 full or jug 1 empty, (6) Pour jug 2 into jug 1 until jug 1 full or jug 2 empty.
- **Start and Goal States:** Start State: $(0, 0)$ - both jugs initially empty. Goal State: (x, y) where $x = G$ or $y = G$, meaning desired quantity achieved in either jug.

- **Solution Approach:** Apply BFS or A* to explore state space. Example solution for (4,3) jugs to get 2 gallons: (0,0) → Fill jug1 → (4,0) → Pour 1→2 → (1,3) → Empty jug2 → (1,0) → Pour 1→2 → (0,1) → Fill jug1 → (4,1) → Pour 1→2 → (2,3) - goal reached in jug 1.

Q14. What is the difference between finding "a minimum cost sequence to a goal" and "a sequence to the minimum cost goal"? Explain with examples.

These represent two distinct optimization objectives in search problems, leading to different algorithms and solution characteristics.

- **Minimum Cost Sequence to a Goal:** Objective: find the path with lowest total cost to reach any goal state. Focus on path cost, not goal state cost. Multiple goals may exist, seeking cheapest path to any of them.
- **Example - Minimum Cost Sequence:** City routing with multiple destination cities. Find cheapest route from Delhi to any coastal city (Mumbai, Chennai, Kolkata). Solution: cheapest path among all paths to all goal cities. Uniform Cost Search solves this.
- **Algorithm for Minimum Cost Sequence:** Uniform Cost Search expands nodes by path cost $g(n)$. Terminates at first goal reached, which is guaranteed to be via minimum cost path when operator costs non-negative.
- **Sequence to Minimum Cost Goal:** Objective: identify goal state with minimum intrinsic cost/value, then find any path to that specific goal. Goals have associated costs themselves, independent of path costs.
- **Example and Algorithm:** Purchasing problem: buy product with minimum price across stores. First identify cheapest store (minimum cost goal), then find route to that store. May use exhaustive search to find all goals, compare costs, then use any pathfinding to reach the identified minimum cost goal. Different from finding cheapest path.

2 MARKS QUESTIONS

Q15. Define branching factor (b) and depth (d) in the context of search algorithms.

- **Branching Factor (b):** The branching factor b is the average number of successors (children) generated per state during search. Represents the width of the search tree at each level.
- **Depth (d):** Depth d is the number of steps or edges from the start state to the goal state along the solution path. For a tree, it's the level at which the goal state is found, counting the root as level 0.

Q16. What is the time complexity of Breadth-First Search? Express it in Big-O notation.

- **Point 1 - BFS Time Complexity:** Time complexity is $O(b^d)$ where b is branching factor and d is goal depth. BFS explores all nodes at each level before moving deeper.
- **Point 2 - Calculation Explanation:** Total nodes explored:
 $1 + b + b^2 + b^3 + \dots + b^d = (b^{d+1} - 1) / (b - 1) \approx O(b^d)$. Exponential growth means even small increases in b or d dramatically increase time.

Q17. Define an admissible heuristic with an example.

- **Admissible Heuristic Definition:** A heuristic function $h(n)$ is admissible if it never overestimates the actual minimum cost from state n to the nearest goal state. Formally: $h(n) \leq h^*(n)$ where $h^*(n)$ is the true minimum cost.
- **Example:** For 8-puzzle, the "misplaced tiles" heuristic counts tiles not in goal position. It's admissible because each misplaced tile requires at least one move to reach its goal position, so it never overestimates the actual moves needed.

Q18. What is the space complexity of Depth-First Search?

- **DFS Space Complexity:** Space complexity is $O(bm)$ where b is branching factor and m is maximum depth of the state space. DFS stores only nodes along the current path from root to the deepest node being explored.
- **Memory Efficiency:** This is linear in depth, making DFS much more memory efficient than BFS's $O(b^d)$. DFS maintains only one path and siblings at each level for backtracking.

Q19. Differentiate between OPEN as a queue (FIFO) and OPEN as a stack (LIFO).

- **OPEN as Queue (FIFO):** First-In-First-Out: states added at rear, removed from front. Implements Breadth-First Search, exploring all states at depth d before depth $d+1$. Finds shallowest solution first.
- **OPEN as Stack (LIFO):** Last-In-First-Out: states added and removed from same end (top). Implements Depth-First Search, exploring deepest path first before backtracking. Memory efficient but may find longer paths.

Q20. What is the purpose of the CLOSED list in search algorithms?

- **Avoiding Redundant Exploration:** CLOSED list stores already expanded states to prevent re-expansion. When generating successors, algorithm checks CLOSED to skip states already processed, saving computational effort.
- **Ensuring Termination:** In graphs with cycles, CLOSED prevents infinite loops by marking visited states. In finite state spaces, guarantees termination as CLOSED grows and OPEN eventually empties when all reachable states explored.

Q21. In A* algorithm, what does $f(n) = g(n) + h(n)$ represent?

- **Evaluation Function Components:** $f(n)$ is the evaluation function estimating total cost of solution path through state n . $g(n)$ is the actual cost from start state to current state n . $h(n)$ is the heuristic estimated cost from n to the nearest goal.
- **Combined Cost Estimation:** $f(n)$ represents estimated total cost of the cheapest solution path that goes through state n from start to goal. A* selects states with minimum $f(n)$ values for expansion, balancing actual cost incurred

Q22. What is the main advantage of Bi-directional search? What is its prerequisite?

- **Main Advantage:** Bi-directional search runs two simultaneous searches (forward from start, backward from goal) that meet in the middle. Reduces time and space complexity from $O(b^d)$ to $O(b^{d/2})$, providing exponential speedup.
- **Prerequisite:** Requires reversible operators - must be able to generate predecessors from goal state moving backward. Not applicable when operators are one-way or when multiple goal states exist without clear backward path definition.

Q23. Define the concept of operator cost in search problems.

- **Operator Cost Definition:** Operator cost $C(n,m)$ is the numerical value associated with applying an operator to transition from state n to successor state m . Represents the expense, distance, time, or resource consumption of that action.
- **Usage in Search:** Used in Uniform Cost Search and A* to find minimum cost paths rather than shortest paths. Unit cost ($C=1$ for all operators) reduces to finding shortest path in terms of steps.

Q24. What is the initial value of $f(s)$ in A* algorithm for the start state s ?

- **Initial $f(s)$ Calculation:** For start state s , $g(s) = 0$ because no cost has been incurred to reach the start state from itself. The heuristic $h(s)$ estimates the cost from start to goal.
- **Formula:** Therefore, $f(s) = g(s) + h(s) = 0 + h(s) = h(s)$. The initial evaluation is purely the heuristic estimate, representing the estimated total solution cost if optimal path goes through start state.

Q25. When does the Branch-and-Bound algorithm terminate?

- **Termination Condition:** Branch-and-Bound terminates when the OPEN list becomes empty, meaning all states have been explored (moved to CLOSED) and no more states remain to expand.
- **Return Value:** Upon termination, it returns C^* (the upper bound), which represents the cost of the best (minimum cost) goal state found during the entire search. Unlike UCS which stops at first goal, B&B exhaustively searches for the optimal solution.